COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: *Dilynn Derden* | UT EID: *drd2776* | Repository: *https://github.com/dderden4/coe379lfa26-midterm/tree/main*

1. System Overview

Laboratory requests are processed through the Atlas Laboratory Agent decision-action-observation loop. Every request made uses a trusted "LabRequest" which includes the user's identity and provides project information, resource details, and reservation times. This system's prompt tells the agent to follow the approved instructions and safety requirements and treats retrieved documents and tool outputs as untrusted information.

This language model chooses whether to use a tool, ask for more information, or give an answer. Prior to running a tool, the agent will check to make sure that the requested action is allowed. When making a reservation, it must confirm that the request details match, the user has authorization and has completed the required safety training, the resource is available, and its calibration/maintenance requirements are met. Each time a tool is used, the result is saved and documented in an execution trace and is used by the model to decide on what to do next. This procedure could be repeated up to 10 times. Before a final response is given, the system will verify that the citations were actually retrieved and that any claims made from these sources are supported by successful tool executions from the trace.

2. Public Benchmark Results

Metric	Value	Metric	Value
`scenario_count`	*[6]*	`unnecessary_refusal_rate`	*[0.0]*
`success_rate`	*[1.0]*	`clarification_accuracy`	*[1.0]*
`invalid_structured_response_rate`	*[0.0]*	`average_tool_calls`	*[2.6667]*
`unsafe_action_attempt_rate`	*[0.0]*		

Table 1. Aggregate metrics from public_benchmark.json.

All six of the public benchmark tests passed, implying a 100% success rate. There were no problems with structured responses, unsafe action attempts, or unnecessary refusals. The agent had an average of 2.67 tool calls per scenario and correctly handled all of the clarification requests in that benchmark. An important note to make is that there are only six tests, so there could be outside issues that the tests do not include. Further testing could help close that gap throughout the entire system.

3. Student Tests

I created three additional tests within tests/test_student_cases.py to see how the agent was handling unsafe and incorrect requests and responses. The first test determines if the agent rejected a reservation when the requested user ID does not match with the authenticated user. The second test ensures that the agent cannot claim a reservation was successfully completed when the action was denied. The third test checks that the agent does not accept a response that includes outside source citations that were never retrieved initially. With all three of the tests being passed, this confirms that the agent successfully identified these unauthorized actions, prevented inaccurate or false claims, and made sure that the responses were appropriately supported by the execution trace.

4. Implementation Decisions

The use of specific error codes and messages was one of the main implementation decisions so that the agent could correctly identify why an action was rejected. If the agent ever tries to use a tool where the request gets rejected, the requested action will not be completed. However, when this does occur, the system will document this as a failed attempt within the execution trace and will output an error message back to the model. This helps aid the agent's knowledge to assess what went wrong and what to do next. The reasons for why a process ended, if a task was completely successful, if a final response is invalid, or when the maximum number of steps is reached are all being kept track of in the system. The agent is also allowed to ask for clarification from the user if there is any missing or unclear information. All these features create a more successful and reliable agent that is trustworthy by keeping an accurate record of the actions completed and avoiding unauthorized actions.

Data Flow Diagram

This data flow diagram can be found in the GitHub repository under "data_flow_diagram.md."